



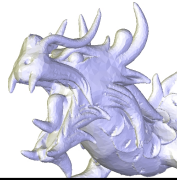
Lyon 1

Meshes and Computational Geometry

Raphaëlle Chaîne
Université Claude Bernard Lyon 1

M2 ID3D
Image, Développement
et Technologie 3D
et 3A Centrale

2026-27



1

Which place for Geometry in Computer Science?

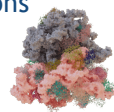
- Geometric problems and applications
 - Computer Graphics
 - Collision detection
 - Illumination calculation
 - Hidden parts not taken into account
 - Robotics and 3D vision
 - Trajectory planning
 - Telecommunication
 - Determination of the nearest relay

2

2

Which place for Geometry in Computer Science?

- Geometric problems and applications
 - Chemistry
 - Molecular modeling
 - Pocket detection
 - Calculation of contact surfaces
 - CAD (Computer Aided Design), CAM (Computer Aided Manufacturing)
 - Integrated circuit design
 - *Reverse engineering*
 - Scientific Computation : physics, geology



C. Plateau Holleville



3

3

Which place for Geometry in Computer Science?

- Geometric problems and applications
 - Digital twins
 - Museums and virtual shops
 - Digitization and availability of cultural heritage
 - Geographic Information Systems (GIS)
 - Virtual creation
 - Animated films, video games
 - Digital entertainment

4

4

Difficulty of dealing with geometry in computer science...

5

5

Nuance between Computational Geometry and Discrete Geometry approaches

- Common goal
 - Dealing with geometric objects in a discrete world
- Discrete Geometry
 - Discretization of geometric objects on grids
1 point = 1 pixel



6

6

Nuance between Computational Geometry and Discrete Geometry approaches

- Computational geometry
 - Handling geometric objects as abstract type instances
 - Operations consistent with the geometric properties of the objects being processed

7

7

Nuance between Computational Geometry and Discrete Geometry approaches

- Computational Geometry
 - Combinatorial objects
 - The shapes are described by assembly of geometric primitives
 - Everything can be built combinatorically on the notion of vertex

8

8

What elementary geometric objects do you think you need to model first?

9

9

What elementary geometric objects do you think you need to model first?

- Points :
 - Which points?
- Remember that we are subject to discrete arithmetic...

Floating point real representation

$(-1)^s \cdot 2^{(\text{exponent} - \text{offset})} \cdot 1, \mathbf{M}$ (norme IEEE)

Simple precision : exp 8 bits, M 23-1 bits, offset=127

Double precision : exp 11 bits, M 52-1 bits, offset=1023¹⁰

10

What elementary geometric objects do you think you need to model?

- Points
- Segments  can be used to make broken lines
- How would you model segments?
- What is the intersection of two segments?
- Vectors, straight lines, half straight lines, ...

11

11

What elementary geometric objects do you think you need to model?

- Back to the notion of point:
 - A triplet of coordinates
 - OR A construction tree!

12

12

What elementary geometric objects do you think you need to model?

- Polygons
 - Triangle = polygon determined by a minimum number of points
- Cercles, Spheres
- Many shapes can be described combinatorically from a few points

13

13

What elementary geometric objects do you think you need to model?

- What if we work in a world with dimensions greater than 2 or 3?

14

14

Concept of affine combination

$$p = \sum_{i=1}^n \lambda_i x_i, \quad \sum_{i=1}^n \lambda_i = 1$$

- Affine combination of n points
 - All possible combinations of these n points
 - The coefficients λ_i can be negative or positive
- Example :
 - All the points of a line AB are expressed by affine combination of A and B

15

15

Concept of convex envelope

$$p = \sum_{i=1}^n \lambda_i x_i, \quad \sum_{i=1}^n \lambda_i = 1$$

- Convex envelope of n points :
 - All convex combinations of these n points: affine combinations obtained with **positive coefficients** λ_i
- Coefficients λ_i are denoted as barycentric coordinates
 - **Unique if** the points are independent (implies a number of points lower than the dimension of the space + 1)

16

16

Concept of affine combination

$$p = \sum_{i=1}^n \lambda_i x_i, \quad \sum_{i=1}^n \lambda_i = 1$$

- Two points :
 - Affine combination : straight line
 - Convex combinaison (positive coefficients) : segment
- Three points
 - Affine combination : plane
 - Convex combination : triangle
- Concept of barycentric coordinates

17

17

Vocabulary used in Computational Geometry

- 0-simplex : point
- 1-simplex : segment
- 2-simplex : triangle
- 3-simplex : tetrahedron
- k-simplex : Convex envelope of k+1 points being independents (affine combinations of these points = space of dimension k)

18

18

Concept of barycentric coordinates

$$p = \sum_{i=1}^n \lambda_i x_i, \quad \sum_{i=1}^n \lambda_i = 1$$

- Barycentric coordinates of a point Q inside a simplex $D = \{P_{b0}, P_{b1}, \dots, P_{bN}\}$
- Coordinate wrt P_{bj}
 - Let F_{bj} be the face opposite to P_{bj} ,
 - $\lambda_j = \text{Volume}(Q, F_{bj}) / \text{Volume}(D)$
 - $\text{Volume}(D) = \text{determinant}$

$$V(D) = \frac{1}{N!} \det(P_{b1} - P_{b0}, P_{b2} - P_{b0}, \dots, P_{bN} - P_{b0})$$

19

19

Volume in 2D and in 3D?

- Determinant linked to :
 - Cross-product of 2 vectors : use symbol $\times$
 - Direction? Length?
 - Dot product of 2 vectors
- Area in 2D
Area(abc) liée à $1/2 (b-a) \times (c-a)$
- Volume in 3D
Volume(abcd) = $1/3 \text{ Area}(\text{base}) \times \text{height}$

20

20

Volume in 2D and in 3D?

- Volume(abcd) = Volume(T, d) T=triangle(abc)

$$\begin{aligned} \text{Height}(T, d) &= \langle d - a, \vec{n}_T \rangle \\ &= \left\langle d - a, \frac{(b-a) \times (c-a)}{\|(b-a) \times (c-a)\|} \right\rangle \\ \text{Volume}(T) &= \frac{1}{3} \cdot \text{base} \cdot \text{height} \\ &= \frac{1}{6} \times \langle d - a, (b-a) \times (c-a) \rangle \end{aligned}$$

21

21

Vocabulary used in Computational Geometry

- Face : The **faces of a k-simplex** defined by $k+1$ points are the d -simplexes that can be formed from a subset of $d+1 < k+1$ of its vertices
- Given a set of points in a d -dimensional space, it is possible to decompose the space into d -simplexes : **triangulation**

22

22

Vocabulary used in Computational Geometry

- Definition of triangulation :

Given a set E of points of $\mathbb{R}^k$, we call triangulation of E a set of **k-simplexes** whose vertices are the points of E and verifying :

- The intersection of 2 k -simplexes is either empty or a face common to the 2 k -simplexes,
- The k -simplexes pave the convex envelope of E .

23

23

Triangulation

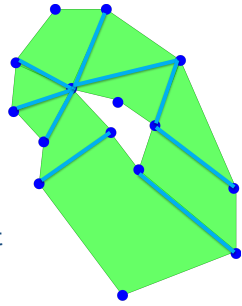
- For the sake of simplicity, we still speak of triangulation in the case where all the k -simplexes rely on vertices in a space with dimension larger than k
- The coverage constraint is then released
- Example :
Triangulation of a surface ($k=2$) based on 3D points

24

24

Combinatorics of connected 2D meshes

- Is there a connection between
 - the number of cells,
 - the number of edges
 - and the number of holes in this mesh ?
- To be noted: the cells are not necessarily triangles!

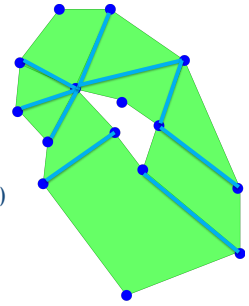


25

25

Combinatorics of connected 2D meshes

- Euler's relationship
Planar triangulation
 $c - a + s = 1 - t$
 - c : number of cells
 - a : number of edges
 - s : number of vertices ($s > 1$)
 - t : number of holes



26

26

Combinatorics of connected 2D meshes

- Euler's relationship, Planar mesh
 $c - a + s = 1 - h$ (h holes)
- Intuition of the proof
 - We start from a single vertex (Euler's relationship is checked: $0 - 0 + 1 = 1 - 0$) and we add the edges one by one by maintaining one connected component.
 - Add an edge based on a new vertex: $s + = 1$, $a + = 1$
 - Add an edge based on existing vertices: $c + = 1$ or $h + = 1$
 - At each addition of an edge, $(c + h) - a + s$ remains invariant.

27

27

Combinatorics of connected 2D meshes

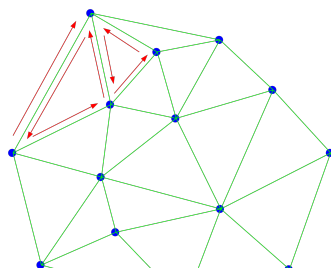
- Euler's relationship
Polyhedra : Surface mesh of a closed 3D shape
 $c - a + s = 2(1 - g)$
 - c : number of cells
 - a : number of edges
 - s : number of vertices ($s > 1$)
 - g : genus of the surface (number of tunnels)

28

28

Special case of triangulations

- Can we say more about the link between the number of faces, edges and vertices?

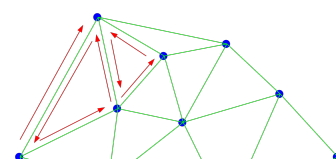


29

29

Special case of triangulations

- No hole ($t = 0$)
- Each cell is bordered by 3 edges
- $2a = 3c + k$
 - where k is the number of edges on the convex envelope (ie. the edge)



30

30

Special case of triangulations

- Number of cells and edges of a triangulation of s points
 - Euler's relationship:
 - $c - a + s = 1$
 - Relationship between a and c in a triangulation :
 $2a = 3c + k$ where k is the number of edges on the convex envelope
 - $c = 2s - 2 - k$
 - $a = 3s - 3 - k$

31

31

Data structures

- How to store a triangular mesh so that one can easily navigate through it?

32

32

Data structures

- Geometric information :
 - coordinates of vertex positions
- Topological information :
 - incidence and adjacency relationships between vertices, edges and faces
- Access to an information :
Address or index in a table

33

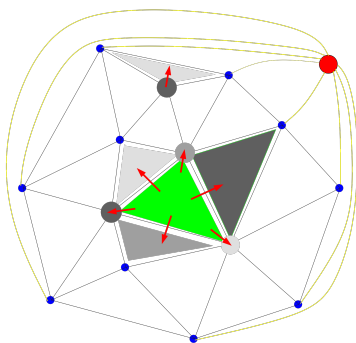
33

Data structures

- Representation based on faces and vertices
(for triangulated meshes only)
 - Triangle :
 - Access to the 3 incident vertices (trigonometric order)
 - Access to the 3 adjacent triangles
Constraint : vertex i facing adjacent triangle i
 - Vertex :
 - Access to 1 incident triangle
 - Access to the underlying point (geometry)

34

34



35

35

- What is the difference between the **global index** of a vertex, and its **relative index** in a face?

36

36

Data structures and boundaries

- Case of a 2D triangulation
 - Dangling pointers / indexes on the boundary of the convex hull
- OR
 - Addition of a fictitious vertex (called infinite vertex) whose incident triangles are attached to the edges of the convex envelope
- The data structure can also be used for surface triangulations
 - Use dangling pointers/indexes for each hole boundary
- OR
 - Add fictitious vertices for each hole

37

37

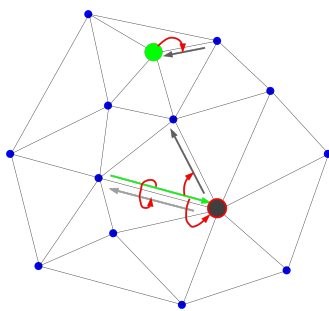
Data structures with edges

- Representation based on $\frac{1}{2}$ edges and vertices
 - $\frac{1}{2}$ edge :
 - Access to the coupled $\frac{1}{2}$ edge
 - Access to the next $\frac{1}{2}$ edge
 - Access to the target vertex
 - Vertex :
 - Access to an $\frac{1}{2}$ edge oriented towards the vertex
 - Access to the underlying point

38

38

Data structures



39

39

Naive and uncompressed format of a mesh in a file

- Files off
- Format description
 - Number of vertices s
 - Number of faces c
 - Vertices coordinates
 - Description of faces (from 0 to $c-1$)
 - Number of vertices in the face
 - Description of the face by the indexes of its vertices (in counter-clockwise direction in 2D, or by orienting the faces towards the outside in 3D)

40

40

How to load a mesh into the data structure?

- Need to find adjacencies between faces
- Using a *temporary map* while reading faces
 - To each edge of a face
(key = pair of vertex indexes)
we associate the *index* of the current face
(+ optionally the relative index of the vertex opposite to the edge in the face)
 - When an edge is incident to two faces, the adjacency between these two faces is reported in the map

41

41

How to load a mesh into a data structure?

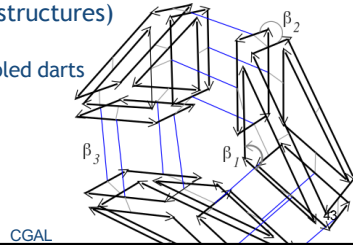
- Using a *map* while reading faces
- Complexity in $O(s \cdot \log(s))$ where s number of vertices
- Rque : If you use an *hmap* instead of a *map*, you get an almost linear complexity.

42

42

Data structures

- Case of a 3D triangulation
 - Structure based on tetrahedrons and vertices
- OR
- Combinatorial maps (extension of half-edge or dart-based structures)
 - β_1 : next
 - β_2 et β_3 : coupled darts



43

Data structures

- Case of nD triangulations
 - n-simplex and vertex based structures
 - Combinatorial Maps
 - β_1 : next
 - $\beta_2, \beta_3 \dots \beta_n$: coupled darts

44

44